

TP5: Driver de caracteres para sensado de señales externas

Materia: Sistemas de Computación

Grupo: Electrotonto y Computarados

Integrantes:

- Martina Juri
- Marcos Morán
- Francisco Gomez Neimann
- Cristian Eduardo Arteaga Barrera

1. Introducción

En este trabajo práctico se diseñó e implementó un **Character Device Driver (CDD)** para Linux, orientado a ejecutarse sobre una Raspberry Pi 3. El objetivo principal fue construir un driver capaz de sensar dos señales externas con un período de muestreo de **un segundo**, exponer dichas mediciones mediante un dispositivo de caracteres y permitir que una aplicación de usuario seleccione cuál de las dos señales desea leer.

Un driver de dispositivo es una pieza de software que permite al sistema operativo administrar, controlar y comunicarse con un periférico. En este caso, el periférico utilizado fueron dos entradas digitales conectadas a los pines GPIO de la Raspberry Pi. El driver se ejecuta en espacio de kernel, mientras que la visualización, presentación y corrección de escala se realizan en espacio de usuario, respetando la separación propuesta por el enunciado.

La aplicación de usuario se implementó como un servidor web en Python. Esta decisión permitió visualizar la señal desde cualquier navegador conectado a la misma red, evitando depender de una interfaz gráfica local en la Raspberry Pi.

2. Objetivos

Los objetivos principales del trabajo fueron:

- Implementar un módulo de kernel cargable (**.ko**) que registre un dispositivo de caracteres.
- Sensar dos señales externas con un período de un segundo.
- Permitir seleccionar desde espacio de usuario cuál señal debe entregar el CDD.
- Exponer las mediciones mediante **/dev/sensor_cdd**.
- Implementar una aplicación de usuario que lea el driver y grafique la señal en tiempo real.
- Resetear el gráfico al cambiar de señal.
- Realizar la corrección de escala y metadata de visualización en espacio de usuario.
- Desplegar la solución completa en una Raspberry Pi 3.

3. Arquitectura general

La solución final quedó organizada de la siguiente manera:

```
Botón 1 / señal externa 1 — GPIO17 ┐
                                     └─ cdd_sensor.ko — /dev/sensor_cdd —
sensor_server.py — navegador
Botón 2 / señal externa 2 — GPIO27 ┘      1 Hz      read/write      HTTP
+ SSE
```

El sistema se divide en dos partes:

- **Kernel:** módulo `cdd_sensor.ko`, encargado de leer los GPIO y exponer un CDD.
- **Usuario:** servidor `sensor_server.py`, encargado de leer `/dev/sensor_cdd`, servir la página web y graficar las muestras.

La comunicación entre la página web y el servidor se realiza mediante **Server-Sent Events (SSE)**. Este mecanismo permite enviar muestras nuevas al navegador en tiempo real sin refrescar la página y sin implementar una aplicación de escritorio pesada.

4. Driver de caracteres

El driver implementado se encuentra en:

```
TP5/kernel/cdd_sensor.c
```

Al cargarse, registra el dispositivo:

```
/dev/sensor_cdd
```

La interfaz implementada es:

- `read()`: bloquea hasta que exista una muestra nueva y devuelve el valor como texto.
- `write()`: recibe `1` o `2` para seleccionar la señal activa.
- `ioctl()`: permite consultar o modificar la señal activa y obtener metadata básica.
- `timer_list`: genera el muestreo periódico cada un segundo.
- `kfifo`: almacena muestras pendientes para la lectura desde usuario.

El driver toma una muestra por segundo. Si el FIFO se llena, descarta la muestra más vieja para conservar datos recientes. Cuando se cambia de señal, el FIFO se limpia para evitar mezclar muestras de señales distintas.

5. Hardware utilizado

Para la prueba se utilizaron dos pulsadores como señales digitales externas. Cada botón representa una señal binaria:

- Sin presionar: `0`
- Presionado: `1`

Conexión física utilizada:

Señal	GPIO BCM	GPIO global en esta Raspberry	Pin físico Raspberry Pi 3
Señal 1	GPIO17	529	11
Señal 2	GPIO27	539	13
3.3 V	-	-	1
GND	-	-	6

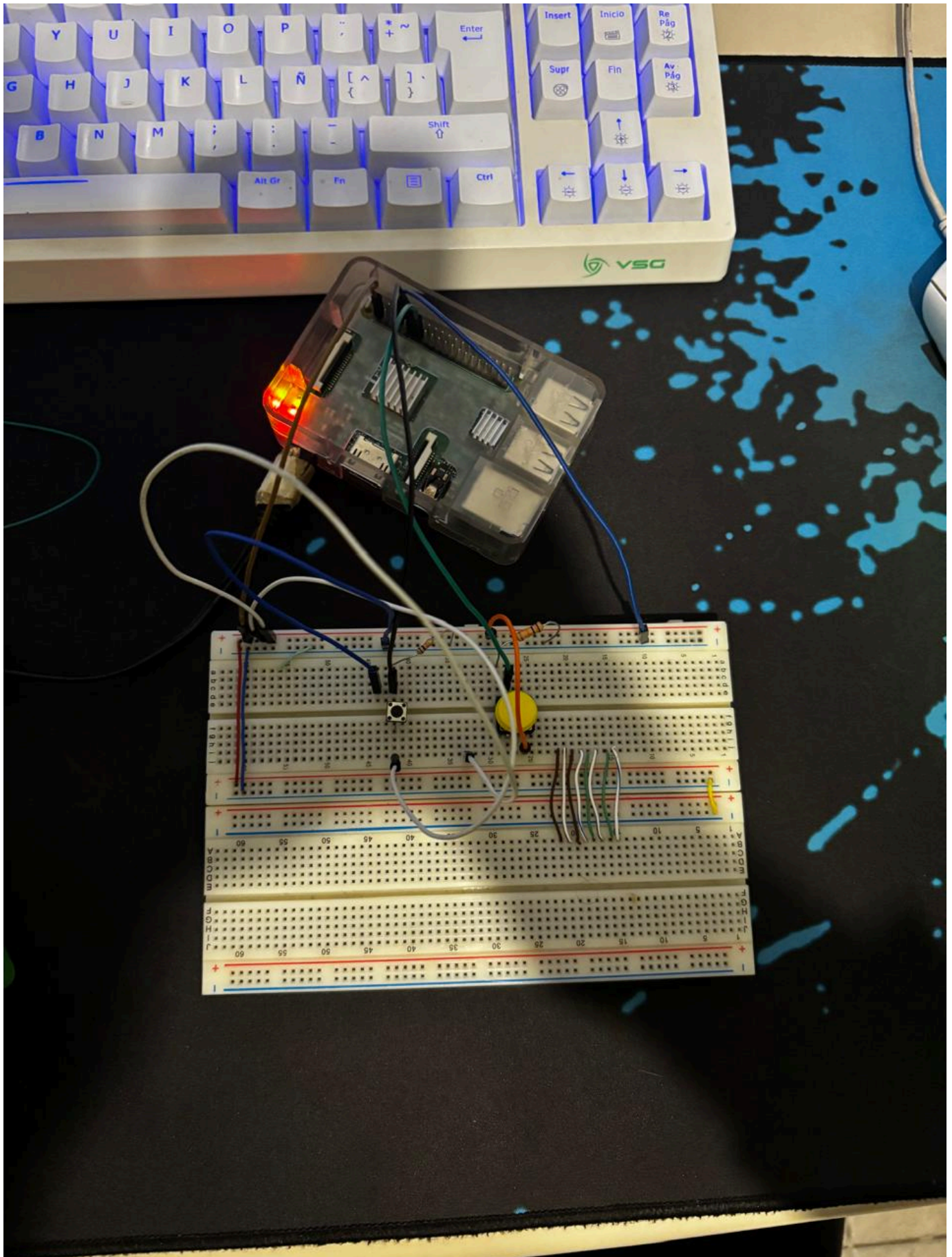
Esquema de conexión recomendado con resistencias pull-down de 10 kΩ:

```
Pin 1 (3.3V) — Botón 1 — Pin 11 (GPIO17)
Pin 11 (GPIO17) — Resistencia 10 kΩ — Pin 6 (GND)
```

Pin 1 (3.3V) — Botón 2 — Pin 13 (GPIO27)
Pin 13 (GPIO27) — Resistencia 10 k Ω — Pin 6 (GND)

Es importante no conectar nunca 5 V directamente a un GPIO, ya que la Raspberry Pi trabaja con niveles lógicos de **3.3 V**.

Circuito armado



6. Particularidad de numeración GPIO

Durante la puesta en marcha se encontró que el kernel de la Raspberry no aceptaba directamente los valores BCM **17** y **27** como números GPIO globales. Al intentar cargar el módulo con:

```
sudo insmod kernel/cdd_sensor.ko gpio1=17 gpio2=27
```

el kernel devolvía:

```
cdd_sensor: no se pudo reservar cdd_sensor_sig1 (GPIO 17): -517
```

La causa fue que el kernel expone los GPIO con una base global distinta. Esto se verificó con:

```
sudo mount -t debugfs none /sys/kernel/debug 2>/dev/null || true
sudo cat /sys/kernel/debug/gpio
```

La salida relevante fue:

```
gpiochip0: GPIOs 512-565
gpio-529 (GPIO17)
gpio-539 (GPIO27)
```

Por lo tanto, en esta Raspberry:

```
GPIO17 BCM -> GPIO global 529
GPIO27 BCM -> GPIO global 539
```

La carga correcta del módulo fue:

```
sudo insmod kernel/cdd_sensor.ko gpio1=529 gpio2=539
```

Esta diferencia es importante porque los pines físicos no cambiaron: GPIO17 siguió siendo el pin físico 11 y GPIO27 siguió siendo el pin físico 13. Lo que cambió fue la numeración global que el kernel utiliza internamente para reservar los GPIO.

7. Compilación

El enfoque propuesto por la cátedra es la compilación cruzada desde la PC host hacia ARM. Para ello se requiere tener instalados los headers o fuentes exactas del kernel que ejecuta la Raspberry Pi.

Ejemplo de compilación cruzada desde la PC:

```
cd TP5
make module \
  ARCH=arm \
  CROSS_COMPILE=arm-linux-gnueabihf- \
  KERNEL_DIR=$HOME/rpi-kernel/linux
```

En la práctica, durante la puesta en marcha se detectó que `KERNEL_DIR` no existía en la PC host:

```
/home/marcos/rpi-kernel/linux: No existe el archivo o el directorio
```

Esto indicó que todavía no estaba preparado el árbol de kernel de Raspberry necesario para la compilación cruzada. Para validar el desarrollo y evitar incompatibilidades de headers, también se realizó una compilación directamente dentro de la Raspberry Pi usando los headers locales:

```
sudo apt update
sudo apt install -y raspberrypi-kernel-headers build-essential

cd /home/marcos/tp5

ARCH_NAME="$(dpkg --print-architecture)"
if [ "$ARCH_NAME" = "arm64" ]; then KARCH=arm64; else KARCH=arm; fi

make module ARCH="$KARCH" CROSS_COMPILE= KERNEL_DIR="/lib/modules/$(uname -r)/build"
```

Este procedimiento permitió compilar el módulo contra el kernel efectivamente instalado en la Raspberry.

8. Despliegue en la Raspberry Pi

Desde la PC host se transfirió el proyecto a la Raspberry mediante `rsync`.

En este caso, el usuario remoto era `marcos`, por lo que el destino correcto fue `/home/marcos/tp5`:

```
rsync -av TP5/ marcos@192.168.1.18:/home/marcos/tp5/
```

Inicialmente se intentó copiar a `/home/pi/tp5`, pero esa ruta no existía porque la Raspberry no usaba el usuario `pi`. El error observado fue:

```
rsync: [Receiver] mkdir "/home/pi/tp5" failed: No such file or directory (2)
```

La solución fue crear y usar la carpeta del usuario real:

```
ssh marcos@192.168.1.18 "mkdir -p /home/marcos/tp5"
rsync -av TP5/ marcos@192.168.1.18:/home/marcos/tp5/
```

9. Carga del módulo

Para cargar el módulo en modo real:

```
cd /home/marcos/tp5

sudo rmmod cdd_sensor 2>/dev/null || true
sudo insmod kernel/cdd_sensor.ko gpio1=529 gpio2=539
```

```
dmesg | tail -20  
ls -l /dev/sensor_cdd
```

Para probar sin hardware conectado, el driver también incluye un modo de simulación:

```
sudo rmmod cdd_sensor 2>/dev/null || true  
sudo insmod kernel/cdd_sensor.ko simulate=1
```

Este modo fue útil para separar errores del driver y de la aplicación web de posibles problemas eléctricos en el cableado.

10. Prueba directa del CDD

Para leer la señal activa:

```
sudo cat /dev/sensor_cdd
```

Por defecto se lee la señal 1. Para cambiar a la señal 2 desde otra terminal:

```
echo 2 | sudo tee /dev/sensor_cdd
```

Para volver a la señal 1:

```
echo 1 | sudo tee /dev/sensor_cdd
```

Durante la prueba con botones se observó inicialmente una alternancia entre 0 y 1 aunque no se presionaran los pulsadores. El problema terminó siendo eléctrico/mecánico en los botones utilizados. Una vez corregido el cableado y el contacto de los pulsadores, el driver comenzó a leer correctamente:

- botón suelto: 0
- botón presionado: 1

11. Aplicación web

La aplicación de usuario se encuentra en:

```
TP5/userapp/sensor_server.py
```

El servidor se ejecuta con:

```
cd /home/marcos/tp5/userapp  
sudo python3 sensor_server.py --device /dev/sensor_cdd --port 8080
```

Luego se accede desde la PC host mediante:


```
http://192.168.1.18:8080
```

La aplicación permite:

- visualizar la señal activa en tiempo real;
- cambiar entre señal 1 y señal 2;
- resetear el gráfico al cambiar de señal;
- mostrar cantidad de muestras recibidas;
- mostrar la hora de la última muestra;
- mantener la visualización en una ventana temporal.

La dependencia principal de Python es Flask:

```
sudo apt install -y python3-flask
```

o alternativamente:

```
pip3 install flask --break-system-packages
```

12. Evidencia de funcionamiento

A continuación se muestran capturas de la aplicación web funcionando con ambas señales.

Señal 1



Señal 2



Video de funcionamiento

El siguiente video muestra la prueba completa con la Raspberry Pi, los pulsadores y la interfaz web actualizándose en tiempo real:

Si el visor Markdown utilizado no reproduce videos embebidos, el archivo puede abrirse directamente desde:

TP5/Multimedia/funcionamiento.mp4

13. Problemas encontrados y resolución

Durante el desarrollo aparecieron varios problemas prácticos.

Headers del kernel inexistentes en la PC host

El primer intento de compilación cruzada falló porque `KERNEL_DIR` apuntaba a una carpeta inexistente:

```
/home/marcos/rpi-kernel/linux
```

Esto mostró la importancia de tener los headers o fuentes exactas del kernel objetivo cuando se compila un módulo fuera del árbol del kernel.

Compilación ejecutada en la máquina equivocada

En una instancia se ejecutó `make module` desde la PC host usando:

```
/lib/modules/6.17.0-29-generic/build
```

Ese kernel correspondía a la PC de escritorio y no a la Raspberry. Además, la ruta local contenía espacios, lo cual produjo errores en Kbuild. Se resolvió moviendo el proyecto a la Raspberry y compilando contra `/lib/modules/$(uname -r)/build`.

Ruta remota incorrecta

Se intentó desplegar a:

```
/home/pi/tp5
```

pero el usuario real de la Raspberry era `marcos`. La ruta correcta fue:

```
/home/marcos/tp5
```

Error al reservar GPIO

El módulo fallaba con:

```
no se pudo reservar cdd_sensor_sig1 (GPIO 17): -517
```

Se resolvió inspeccionando `/sys/kernel/debug/gpio` y usando la numeración global del kernel:

```
GPIO17 -> 529
GPIO27 -> 539
```

Dependencia Flask faltante

Al ejecutar el servidor web, la Raspberry no tenía Flask instalado. Se resolvió instalando el paquete correspondiente:

```
sudo apt install -y python3-flask
```

Problemas en la interfaz web

Durante la integración, el HTML quedó incompleto y parte del CSS se mostraba como texto en pantalla. Se reemplazó el template por una versión completa y autocontenida, logrando una interfaz funcional y clara.

Problema físico con los botones

Finalmente, al probar el hardware real, la señal alternaba aunque no se presionaran los botones. Esto se corrigió revisando el cableado, los pulsadores y las resistencias pull-down.

14. Conclusión

El trabajo permitió integrar conceptos de bajo nivel de Linux con una aplicación de usuario completa. Se implementó un driver de caracteres real, se lo cargó dinámicamente en el kernel de una Raspberry Pi, se expuso un dispositivo bajo `/dev` y se construyó una interfaz web capaz de graficar las muestras en tiempo real.

Además del desarrollo del código, una parte importante del aprendizaje estuvo en la puesta en marcha: compatibilidad de headers, diferencia entre compilación cruzada y compilación nativa, numeración real de GPIO en kernels recientes, dependencias de Python y problemas eléctricos del hardware.

La solución final cumple con los requisitos del enunciado: sensa dos señales externas con período de un segundo, permite seleccionar cuál leer desde espacio de usuario, resetea el gráfico al cambiar de señal y realiza la visualización en una interfaz web accesible desde la PC host.

15. Comandos rápidos finales

Cargar módulo real:

```
cd /home/marcos/tp5
sudo rmmod cdd_sensor 2>/dev/null || true
sudo insmod kernel/cdd_sensor.ko gpio1=529 gpio2=539
```

Probar lectura directa:

```
sudo cat /dev/sensor_cdd
```

Cambiar señal:

```
echo 2 | sudo tee /dev/sensor_cdd
```

Levantar servidor web:

```
cd /home/marcos/tp5/userapp
sudo python3 sensor_server.py --device /dev/sensor_cdd --port 8080
```

Abrir desde la PC:

```
http://192.168.1.18:8080
```